Modern LangChain Text Loading with Custom BaseLoader

1. Why Is langchain_community Being Sunset?

langchain_community was a large package containing hundreds of integrations such as:

- Document loaders
- Vector stores
- Tools
- Third-party integrations
- Other supporting components

Over time, maintaining all these integrations inside one common package became difficult.

2. Main Reasons for the Sunset

1. Heavyweight Package

The package contained a very large number of integrations.

Different integrations required different optional dependencies.

Conceptually:

langchain_community

↓

Hundreds of Integrations

↓

Many Optional Dependencies

↓

Large Package Structure

2. Dependency Complexity

Different integrations depended on different third-party libraries.

Example:

PDF Loader

↓

PDF Library

Database Integration

↓

Database Library

Vector Store

↓

Vector Database Library

Managing all these dependencies inside one package became complicated.

3. Difficult Maintenance

Maintaining hundreds of unrelated integrations in one package became difficult.

Many Integrations

↓

Many Dependencies

↓

Many Changes

↓

Maintenance Complexity

4. Shared Release Cycle

If one integration required an update, the common package release process could become involved.

Conceptually:

Integration A Changes

↓

Common Package

↓

Shared Release Cycle

Standalone packages allow integrations to evolve more independently.

5. Better Alternative — Standalone Integration Packages

Standalone packages allow:

Independent Development

+

Independent Maintenance

+

Independent Releases

According to the supplied notes:

LangChain officially announced the sunset on 22 May 2026.

3. Important Clarification

Do not misunderstand the word:

Heavyweight

It does **not** mean:

Python loads every library into memory whenever we import langchain_community.

The main issue is related to:

Large Package Structure

↓

Dependency Management

↓

Maintenance Complexity

4. Modern Text File Loading in LangChain

There are two useful approaches.

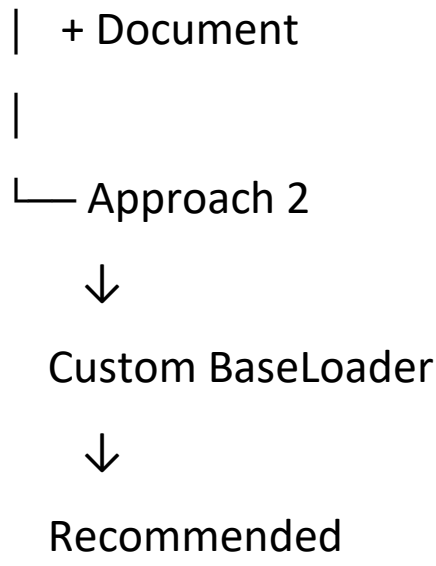
TEXT FILE LOADING

│

├── Approach 1

│ ↓

│ Normal Python



5. Approach 1 — Direct Text Reading and Document Creation

We can use normal Python to read the file and manually create a LangChain Document.

```
from pathlib import Path


from langchain_core.documents import Document



file_path = Path(
    "student_rules.txt"
)
```

```
document = Document(
    page_content=file_path.read_text(
        encoding="utf-8"
    ),

    metadata={
        "source": str(file_path)
    }
)



print(
    document.page_content
)



print(
    document.metadata
)
```

6. How Approach 1 Works

student_rules.txt

↓

Path.read_text()

↓

Python String

↓

Document(...)

↓

LangChain Document

The actual text goes into:

page_content

The source path goes into:

metadata

7. Limitation of Approach 1

Approach 1 uses normal Python without a formal LangChain loader abstraction.

For small examples, this is simple.

But according to these notes, it is not recommended for large-scale applications mainly because of:

1. Scalability Issues

As the application grows, manually managing many loading operations may become difficult.

2. Lack of Standardization

There is no common reusable loader structure.

Conceptually:

Manual Python Reading

↓

Works

↓

But Every Application

May Implement Loading Differently

8. Approach 2 — Creating a LangChain-Compatible Loader

Recommended Approach

We can build our own loader using:

Document

and:

BaseLoader

This follows LangChain's document-loading structure.

9. Advantages of a Custom BaseLoader

1. Scalability

The loader can be extended as the application grows.

2. Standardization

It follows LangChain's standard loading interface.

3. Compatibility

It works smoothly with other LangChain components.

4. Reusability

The same loader can be reused in different applications.

Simple:

Custom Loader

↓

Reusable

↓

Standard

↓

LangChain-Compatible

10. What Is BaseLoader?

BaseLoader is the core interface used to implement LangChain document loaders.

Important import:

```
from langchain_core.document_loaders import (
    BaseLoader
)
```

Our custom loader can inherit from:

BaseLoader

11. What Do We Implement?

In our custom loader, we implement:

lazy_load()

The BaseLoader interface already provides:

load()

So conceptually:

Our Responsibility

↓

lazy_load()

BaseLoader Provides

↓

load()

12. lazy_load() vs load()

Method	Purpose
lazy_load()	Yields Document objects through an iterator
load()	Collects the yielded Documents into a list

Easy memory:

lazy_load()

↓

YIELD

load()

↓

LIST

13. Understanding yield Before Custom Loaders

Before understanding lazy_load(), we should understand Python generators.

Example:

def numbers():

```
    yield 10

    yield 20

    yield 30
```

Now:

```
result = numbers()
```

Check the type:

```
print(
    type(result)
)
```

Output:

```
<class 'generator'>
```

Now iterate:

```
for num in result:

    print(
        num
    )
```

Output:

10

20

30

14. What Does yield Do?

yield returns values one at a time.

Conceptually:

Function Starts

↓

yield 10

↓

Pause

↓

yield 20

↓

Pause

↓

yield 30

Unlike a normal return, yield allows the function to continue later.

15. Normal Function vs Generator

Normal Function

```
def number():

    return 10
```

Concept:

return

↓

Give Value

↓

Function Ends

Generator

```
def numbers():

    yield 10

    yield 20

    yield 30
```

Concept:

yield

↓

Give One Value

↓

Pause

↓

Continue Later

16. Why Is yield Useful for Document Loaders?

A loader may need to produce:

Document 1

Document 2

Document 3

...

Instead of collecting everything immediately, it can:

Yield Document 1

↓

Yield Document 2

↓

Yield Document 3

This matches the design of:

lazy_load()

17. Understanding the Custom Text Loader

We will create our own class:

SimpleTextLoader

Important:

SimpleTextLoader is our own classroom implementation. It is not a built-in LangChain class.

It accepts:

file_path

encoding

It will:

1. Read the specified text file.
2. Create a LangChain Document.
3. Store the source path in metadata.
4. Yield the Document.

18. Program — Creating a Custom LangChain Text Loader

Objective

Create a reusable text loader using the BaseLoader interface.

```
from pathlib import Path

from langchain_core.documents import (
    Document
)

from langchain_core.document_loaders import (
    BaseLoader
)



# ========================================
# CUSTOM TEXT LOADER
# ========================================

class SimpleTextLoader(
    BaseLoader
```

```
):

    def __init__(
        self,
        file_path,
        encoding="utf-8"
    ):

        self.file_path = Path(
            file_path
        )

        self.encoding = encoding


    def lazy_load(self):

        content = (
            self.file_path.read_text(
                encoding=self.encoding
```

```
            )
        )


        yield Document(
            page_content=content,

            metadata={
                "source":
                str(self.file_path)
            }
        )


# ========================================
# CREATE SAMPLE FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
```

```
    / "library_rules.txt"
)


file_path.write_text(
    "Library opens at 8 AM.\n"
    "Library closes at 6 PM.\n",
    encoding="utf-8"
)


# ========================================
# USE CUSTOM LANGCHAIN LOADER
# ========================================

loader = SimpleTextLoader(
    file_path=file_path
)
```

```
documents = loader.load()


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "RETURN TYPE:",
    type(documents).__name__
)


print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


print(
```

```
    "DOCUMENT TYPE:",
    type(documents[0]).__name__
)


print(
    "\nCONTENT:"
)

print(
    documents[0]
    .page_content
    .strip()
)


print(
    "\nSOURCE:"
)
```

```
print(
    Path(
        documents[0]
        .metadata["source"]
    ).name
)
```

19. Understanding SimpleTextLoader

The class starts with:

```
class SimpleTextLoader(
    BaseLoader
):
```

This means:

SimpleTextLoader

↓

Inherits From

↓

BaseLoader

So our loader follows the LangChain loader interface.

20. Understanding __init__()

```
def __init__(
    self,
    file_path,
    encoding="utf-8"
):
```

The loader accepts:

file_path

↓

Which File?

and:

encoding

↓

How to Decode the File?

21. Converting file_path to a Path Object

```
self.file_path = Path(
    file_path
)
```

This gives us a Path object.

Then we can easily use methods such as:

.read_text()

22. Understanding lazy_load()

def lazy_load(self):

This method is responsible for producing Document objects.

Inside it:

```
content = (
    self.file_path.read_text(
        encoding=self.encoding
    )
)
```

reads the text file.

Then:

yield Document(...)

produces a Document.

23. Custom Loader Flow

Text File

↓

SimpleTextLoader

↓

lazy_load()

↓

Read File

↓

Create Document

↓

yield Document

24. Understanding the Document Creation

```
yield Document(
    page_content=content,

    metadata={
        "source":
        str(self.file_path)
```

```
    }
)
```

This produces:

```
Document
   |
   ├── page_content
   |      ↓
   |   File Text
   |
   └── metadata
         ↓
      Source Path
```

25. Why Did We Not Implement load()?

We only implemented:

lazy_load()

But later we use:

documents = loader.load()

Why does this work?

Because:

SimpleTextLoader

↓

inherits

↓

BaseLoader

and according to these notes:

BaseLoader

↓

Already Provides load()

So we do not need to write another separate load() method.

26. Relationship Between lazy_load() and load()

Conceptually:

loader.load()

↓

Uses Loader Interface

↓

Collects Documents

Produced by lazy_load()

↓

Returns List

So:

documents = loader.load()

returns:

list

27. Return Types

documents = loader.load()

returns:

List of Documents

Therefore:

type(documents).__name__

gives:

list

and:

```
type(
   documents[0]
).__name__
```

gives:

Document

Easy memory:

documents

↓

list

documents[0]

↓

Document

28. Accessing Content

documents[0].page_content

gives the actual file content.

Example:

Library opens at 8 AM.

Library closes at 6 PM.

29. Accessing Source Metadata

documents[0].metadata[

"source"

]

returns the source file path.

Then:

Path(

documents[0].metadata["source"]

).name

returns only the filename.

Example:

library_rules.txt

30. Program Flow

library_rules.txt

↓

SimpleTextLoader

↓

BaseLoader

↓

lazy_load()

↓

yield Document

↓

load()

↓

List of Documents

↓

documents[0]

↓

page_content + metadata

31. Important Observation

According to the supplied notes:

At the time of execution, this approach does not produce the langchain_community deprecation warning.

Why?

Because this classroom implementation imports from:

langchain_core

rather than the sunset community loader integration.

32. Main Conclusion

BaseLoader allows us to build our own LangChain-compatible document loaders without depending on sunset community integrations.

33. Program — Loading Multiple Text Files

Objective

Load multiple text files into one collection using a custom BaseLoader.

34. Custom Folder Loader

We will create:

FolderTextLoader

This loader will:

Folder

↓

Find *.txt Files

↓

Read Each File

↓

Create Document

↓

Yield Document

35. Program — FolderTextLoader

```
from pathlib import Path

from langchain_core.documents import (
    Document
)

from langchain_core.document_loaders import (
    BaseLoader
)



# ========================================
# CUSTOM DIRECTORY TEXT LOADER
# ========================================

class FolderTextLoader(
```

```
    BaseLoader
):

    def __init__(
        self,
        folder_path
    ):

        self.folder_path = Path(
            folder_path
        )


    def lazy_load(self):

        for file_path in sorted(
            self.folder_path.glob(
                "*.txt"
            )
        ):
```

```
        content = (
            file_path.read_text(
                encoding="utf-8"
            )
        )


        yield Document(
            page_content=content,

            metadata={
                "source":
                str(file_path)
            }
        )


# ======================================
# PREPARE SAMPLE FILES
```

```
# ========================================

folder = (
    Path(__file__).resolve().parent
    / "day2_sample_files"
)


folder.mkdir(
    exist_ok=True
)


files = {

    "admissions.txt":
    "Admissions start on Monday.",

    "library.txt":
    "Library opens at 8 AM.",
```

```
    "student_rules.txt":
    "Students must carry ID cards."
}


for filename, content in files.items():

    (
        folder / filename
    ).write_text(
        content,
        encoding="utf-8"
    )


# ========================================
# LOAD MULTIPLE TEXT FILES
# ========================================
```

```
loader = FolderTextLoader(
    folder
)


documents = loader.load()


# ========================================
# DISPLAY DOCUMENTS
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


for number, document in enumerate(
    documents,
```

```
    start=1
):

    print(
        "\nDOCUMENT",
        number
    )


    print(
        "SOURCE:",
        Path(
            document.metadata[
                "source"
            ]
        ).name
    )


    print(
```

```
        "CONTENT:",
        document.page_content
    )
```

36. Understanding glob("*.txt")

The line:

```
self.folder_path.glob(
    "*.txt"
)
```

means:

Inside This Folder

↓

Find Files Ending With

↓

.txt

Example:

admissions.txt

library.txt

student_rules.txt

are matched.

But:

notes.pdf

students.csv

are not matched.

37. Why Use sorted()?

```
sorted(
   self.folder_path.glob(
      "*.txt"
   )
)
```

helps process the matching files in a predictable sorted order.

Concept:

Matching Files

↓

sorted()

↓

Consistent Order

38. How FolderTextLoader Works

For each file:

```
for file_path in sorted(...):
```

the loader reads:

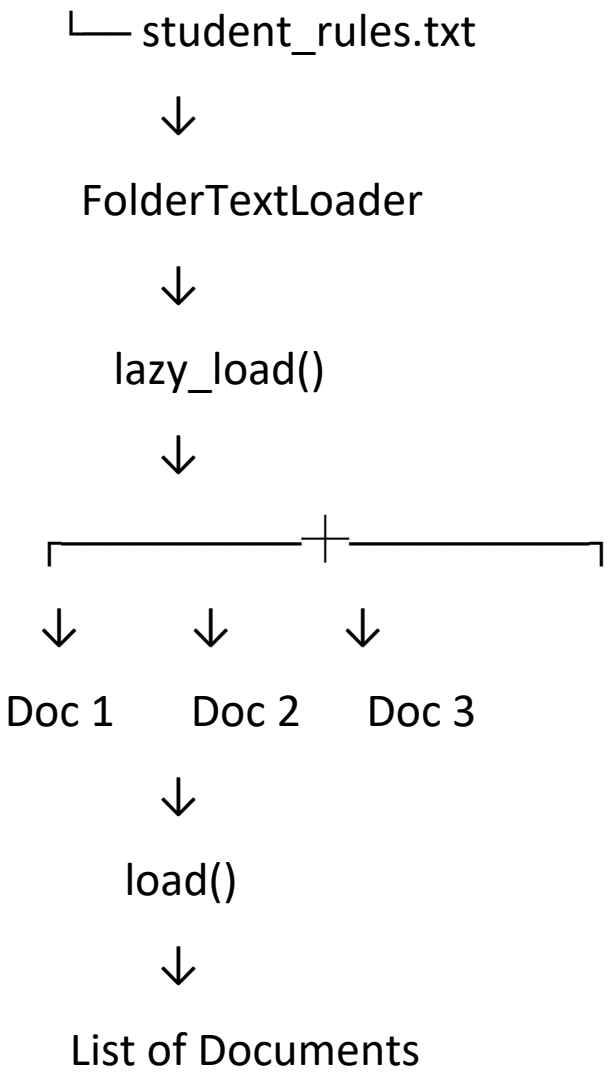
```
content = file_path.read_text(
    encoding="utf-8"
)
```

Then it yields:

```
Document(
    page_content=content,
    metadata={
        "source": str(file_path)
    }
)
```

39. Folder Loader Flow

```
day2_sample_files
        |
        ├── admissions.txt
        ├── library.txt
```



40. Creating the Folder

```
folder.mkdir(
    exist_ok=True
)
```

This creates the folder if needed.

With:

exist_ok=True

the program does not fail simply because the folder already exists.

41. Creating Multiple Files

We use a dictionary:

```
files = {

   "admissions.txt":
   "Admissions start on Monday.",

   "library.txt":
   "Library opens at 8 AM.",

   "student_rules.txt":
   "Students must carry ID cards."
}
```

Then:

```
for filename, content in files.items():
```

creates each file.

42. How Many Documents Are Created?

There are three matching .txt files.

Therefore:

3 Text Files

↓

3 Yields

↓

3 Documents

So:

len(documents)

returns:

3

43. Important Note About Loading Design

One file does not always have to mean exactly one Document.

It depends on how we design our loader.

Example:

One File

↓

Could Produce

↓

One Document

or:

One Large File

↓

Could Be Split

↓

Multiple Documents

So:

The number of Documents depends on the loading design.

44. Important Points

1. Document loading brings external information into LangChain Document objects.
2. The Document class remains in LangChain Core according to these notes.
3. BaseLoader is the interface used here for custom loaders.
4. A custom loader can implement lazy_load() to yield Documents.
5. The inherited .load() method returns the yielded Documents as a list.

6. One file may create one or multiple Documents depending on our design.
7. The classroom examples shown here do not depend on langchain_community.

45. Old Approach vs Modern Classroom Approach

Earlier

TXT File

↓

TextLoader

↓

langchain_community

↓

Document

Current Classroom Approach

TXT File

↓

Custom BaseLoader

↓

langchain_core

↓

Document

46. Approach 1 vs Approach 2

Approach 1	**Approach 2**
Normal Python reading	Custom LangChain loader
Direct Document creation	Inherits BaseLoader
Simple	Structured
Useful for small examples	Better reusable design
No loader abstraction	Standard loader abstraction
Manually manage loading	Implement lazy_load()

47. Most Important Code Structure

```
class MyLoader(
   BaseLoader
):

   def lazy_load(self):

      yield Document(
```

```
        page_content="...",
        metadata={
            "source": "..."
        }
    )
```

This is the core pattern.

48. BaseLoader Master Architecture

External Source

↓

Custom Loader

↓

BaseLoader

↓

lazy_load()

↓

yield Document

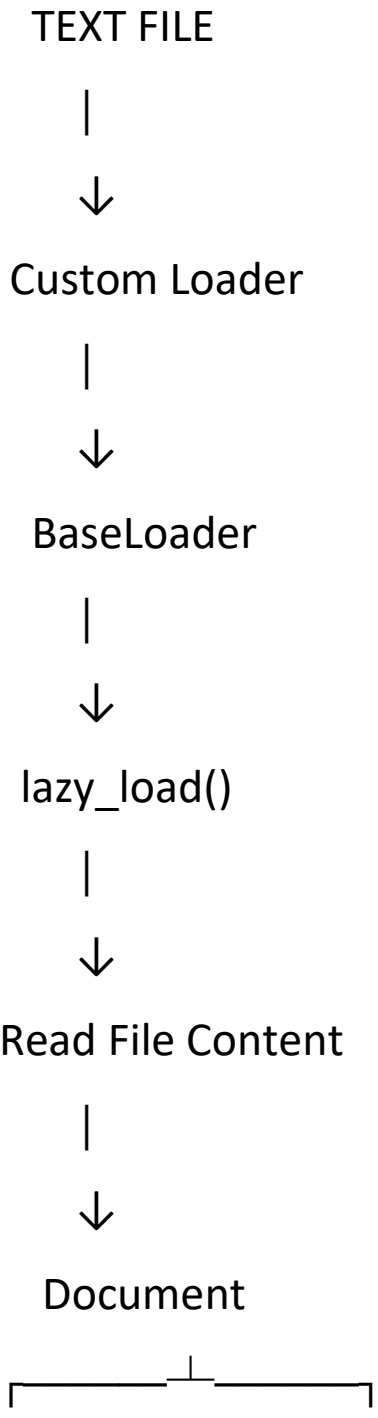
↓

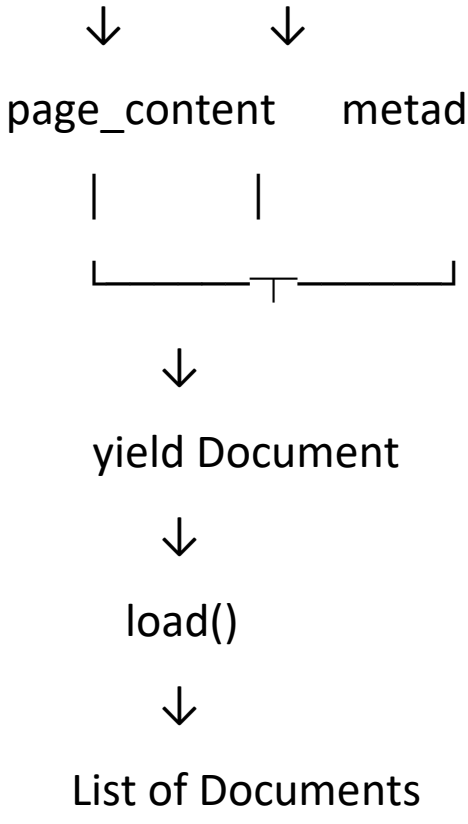
load()

↓

List[Document]

49. Understanding the Complete Flow





50. Common Mistake — Thinking SimpleTextLoader Is Built-In

It is not.

SimpleTextLoader

is:

Our Own Classroom Class

We created it ourselves by inheriting:

BaseLoader

51. Common Mistake — Implementing Both load() and lazy_load() Unnecessarily

In these notes, we implement:

lazy_load()

and use the inherited:

load()

So our classroom pattern is:

Implement

↓

lazy_load()

Inherit

↓

load()

52. Common Mistake — Thinking yield Returns a List

No.

yield

produces values one at a time.

A generator function returns:

generator

not:

list

53. Common Mistake — Confusing lazy_load() with load()

lazy_load()

↓

Iterator / Yielded Documents

load()

↓

List of Documents

Easy memory:

LAZY

↓

ONE BY ONE

LOAD

↓

ALL AS LIST

54. Common Mistake — Thinking BaseLoader Reads Files Automatically

BaseLoader gives us the loader interface.

We still decide:

What Source to Read

How to Read It

How Many Documents to Produce

What Metadata to Store

That logic belongs in our custom loader.

55. Custom Loader Design Formula

Whenever we create a custom loader, think about four things:

1. SOURCE

↓

What are we loading?

2. READ

↓

How do we read it?

3. DOCUMENT

↓

What becomes page_content?

4. METADATA

↓

What source information should we store?

56. Today's Master Formula

TXT File

↓

Custom BaseLoader

↓

Document

↓

List of Documents

More technically:

TXT File

↓

SimpleTextLoader

↓

lazy_load()

↓

yield Document

↓

load()

↓

List[Document]

57. Five Final Revision Points

1. Document

↓

Represents Content + Metadata

2. BaseLoader

↓

Base Interface for Custom Loaders

3. lazy_load()

↓

Yields Documents

4. load()

↓

Returns Documents as a List

5. Custom Loader

↓

Reusable + Standardized

58. One-Line Conclusion

A modern classroom-friendly way to load text files in LangChain is to create our own BaseLoader implementation, implement lazy_load() to yield Document objects, and use the inherited load() method when we need a list of Documents.